DSA Exam Study Guide

Data Structures and Algorithms – CSC211

Tribhuvan University | B.Sc. CSIT – Semester III

Priority-Based Revision | All Units | Repeated Questions

Priority Legend: ■ MUST-KNOW ■ VERY HIGH ■ HIGH ■ MEDIUM

		■ HIGH
		■ MUST-KNOW
Unit 3	Queue	■ MUST-KNOW
Unit 4	Recursion	■ VERY HIGH
Unit 5	Lists (Linked List)	■ MUST-KNOW
Unit 6	Sorting	■ VERY HIGH
Unit 7	Searching & Hashing	■ VERY HIGH
Unit 8	Trees & Graphs	■ MUST-KNOW
—	Exam Strategy & Last-Day Revision Tips	

Unit 1 — Introduction to Data Structures & Algorithms

Syllabus Hours: 4 Hrs ■ HIGH **Appeared in:** 2074, 2075, 2077, 2078, 2079, 2080, 2081

■ Syllabus Topics

- 1.1 Data types, Data structures, Abstract Data Type (ADT)
- 1.2 Dynamic memory allocation in C (malloc, calloc, realloc, free)
- 1.3 Introduction to Algorithms
- 1.4 Asymptotic notations: Big O (worst), Theta (average), Omega (best)

■ Most Repeated Questions (with Year & Section)

#	Question / Topic	Type	Years Asked
1	Describe Big O notation with example / find Big O of f(x)=5x4+9x2+7x+9	Short (B)	2074,2075,2078,2079,2080,2081
2	Explain Theta notation with example	Short (B)	2080
3	What is ADT? Compare data structure with ADT	Short (B)	2075,2077,2078,2079,2080
4	What is dynamic memory allocation? Explain with example.	Short (B)	2075,2077,2079

■ Key Concepts You Must Know

◆ Big O Notation

Defines the upper bound (worst-case) of an algorithm's time/space. Common: O(1) < O(log n) < O(n) < O(n log n) < $O(n^2)$.

◆ Theta (Θ) Notation

Tight bound — algorithm grows exactly at this rate in average case.

◆ ADT vs Data Structure

ADT defines what operations are possible (logical view); Data Structure is the actual implementation (physical view). E.g., Stack is an ADT; Array-based stack is its data structure.

◆ Dynamic Memory Allocation

malloc() — allocates raw memory; calloc() — allocates and zeroes; realloc() — resizes; free() — deallocates. Essential for creating linked list nodes at runtime.

■ *Exam Tip: Big O is asked in almost every exam. Be ready to calculate Big O of a given function (drop constants and lower-order terms). Also know the difference between ADT and Data Structure.*

Unit 2 — Stack

Syllabus Hours: 4 Hrs ■ MUST-KNOW **Appeared in:** 2074, 2075, 2077, 2078, 2079, 2080, 2081

■ Syllabus Topics

- 2.1 Stack concept, Stack as ADT, Push/Pop operations, Applications
- 2.2 Infix → Postfix / Prefix conversion using stack
- 2.3 Evaluation of Postfix / Prefix expressions using stack

■ Most Repeated Questions (with Year & Section)

#	Question / Topic	Type	Years Asked
1	Infix to Postfix conversion using stack (e.g., A+(((B-C)*(D-E)+F)/G)$(H-I))	Both	2074,2075,2078,2079,2081
2	Evaluate postfix expression using stack (e.g., 574-*8/4+)	Short (B)	2074,2077,2078,2079
3	Explain push and pop operations of stack. Applications of stack?	Short (B)	2077,2080
4	Stack as an ADT / Compare stack with queue	Short (B)	2075,2077,2080
5	How does recursive algorithm use stack to store intermediate results?	Long (A)	2079

■ Key Concepts You Must Know

◆ **Stack (LIFO)**

Last-In First-Out. Push adds to top; Pop removes from top. Overflow = push on full; Underflow = pop on empty.

◆ **Infix → Postfix Algorithm**

Scan left to right: (1) operand → output; (2) '(' → push; (3) ')' → pop until '('; (4) operator → pop higher/equal precedence ops first, then push. At end, pop all remaining operators.

◆ **Postfix Evaluation**

Scan left to right: operand → push onto stack; operator → pop two operands, apply operator, push result. Final stack top = answer.

◆ **Applications of Stack**

Expression conversion/evaluation, function call management (call stack), undo/redo, browser back button, Tower of Hanoi.

■ *Exam Tip: Conversion questions always provide a complex infix expression. Practise with operator precedence: ^ > * / > + -. Show each step in a table (symbol, stack state, output) for full marks.*

Unit 3 — Queue

Syllabus Hours: 4 Hrs | ■ MUST-KNOW | **Appeared in:** 2074, 2075, 2077, 2078, 2079, 2080, 2081

■ Syllabus Topics

- 3.1 Queue concept, Queue as ADT, Enqueue/Dequeue operations
- 3.2 Linear Queue and its limitation (memory wastage)
- 3.3 Circular Queue — how it solves linear queue's problem
- 3.4 Priority Queue
- 3.5 Applications of Queue

■ Most Repeated Questions (with Year & Section)

#	Question / Topic	Type	Years Asked
1	Explain Queue as ADT. Write a program to implement linear queue. Compare linear queue vs circular queue.	Long (A)	2080,2079,2077
2	Define circular queue. How does queue differ from stack? Program for linear queue.	Long (A)	2081
3	What is linear queue? Why do we need circular queue?	Short (B)	2079,2081
4	Define queue. Explain enqueue and dequeue in circular queue.	Short (B)	2079
5	What is Priority Queue? Why do we need it?	Short (B)	2075,2078

■ Key Concepts You Must Know

◆ Linear Queue Problem

Even if front elements are deleted, the freed space cannot be reused → apparent overflow. Solved by Circular Queue.

◆ Circular Queue

rear = (rear+1) % SIZE. FULL condition: (rear+1)%SIZE == front. EMPTY: front == -1 or front == rear+1.

◆ Priority Queue

Elements served based on priority, not just arrival order. Used in CPU scheduling, Dijkstra's algorithm, Huffman coding.

◆ Queue vs Stack

Queue: FIFO (First-In First-Out), two ends (front, rear). Stack: LIFO (Last-In First-Out), one end (top).

■ *Exam Tip: For long questions combining queue + program, write the C program with enqueue(), dequeue(), display() functions. Show the circular logic with (rear+1)%SIZE.*

Unit 4 — Recursion

Syllabus Hours: 3 Hrs ■ VERY HIGH **Appeared in:** 2074, 2075, 2077, 2078, 2079, 2080, 2081

■ Syllabus Topics

- 4.1 Principle of Recursion, Recursion vs Iteration, Tail Recursion
- 4.2 Classic Problems: Factorial, Fibonacci Sequence, GCD, Tower of Hanoi (TOH)
- 4.3 Applications and Efficiency of Recursion

■ Most Repeated Questions (with Year & Section)

#	Question / Topic	Type	Years Asked
1	Tower of Hanoi (TOH) — explain with practical example / short note	Both	2074,2078,2081
2	Write a recursive program to find GCD of two numbers	Short (B)	2077,2081
3	Write a recursive program to find nth Fibonacci number	Short (B)	2078
4	Explain Tail Recursion with example. Compare recursion with iteration.	Short (B)	2077,2080
5	How recursive algorithm uses stack to store intermediate results?	Long (A)	2079

■ Key Concepts You Must Know

◆ Principle of Recursion

A function calls itself. Every recursive solution needs a Base Case (termination) and a Recursive Case (reduces problem size).

◆ Tower of Hanoi

Move n disks from source to destination using auxiliary peg. T(n) = 2^n - 1 moves. Recursive solution: move(n-1) to aux, move disk n to dest, move(n-1) to dest.

◆ Tail Recursion

Recursive call is the LAST operation in the function. Can be optimised by compiler into a loop. Regular recursion keeps stack frames; tail recursion reuses them.

◆ Recursion vs Iteration

Recursion: cleaner code, uses call stack (memory), may cause stack overflow. Iteration: more efficient, uses loop variables, preferred for performance.

■ *Exam Tip: GCD and Fibonacci programs are short and scoring. TOH appears as a short note or example. For tail recursion — always contrast it with regular recursion and explain compiler optimisation.*

Unit 5 — Lists (Linked List)

Syllabus Hours: 8 Hrs | ■ MUST-KNOW | **Appeared in:** 2074, 2075, 2077, 2078, 2079, 2080, 2081

■ Syllabus Topics

- 5.1 Basic Concept, List as ADT, Array Implementation of Lists
- 5.2 Singly Linked List — node creation, insertion, deletion (begin/end/position)
- 5.3 Doubly Linked List — insertion/deletion at kth position
- 5.4 Circular Linked List — concept and operations
- 5.5 Stack and Queue implemented using Linked List

■ Most Repeated Questions (with Year & Section)

#	Question / Topic	Type	Years Asked
1	How to implement Stack and Queue using Linked List? Explain.	Both	2079,2078,2077,2080,2081
2	Insert/delete node at kth position in doubly linked list. Algorithm.	Long (A)	2079,2080
3	Differentiate singly and doubly linked list. Insertion/deletion in doubly LL.	Long (A)	2077
4	Explain Circular Linked List with example.	Short (B)	2074,2078,2080,2081
5	What is linked list? How is it different from array?	Short (B)	2075,2077
6	Benefits of linked list over array. Insert a node in singly linked list.	Short (B)	2075

■ Key Concepts You Must Know

◆ Linked List vs Array

Linked List: dynamic size, O(1) insert/delete at known node, O(n) search. Array: fixed size, O(1) random access, O(n) insert/delete. Linked list uses more memory (pointer field).

◆ Singly LL Node

struct Node { int data; struct Node* next; }. Insertion at beginning: newNode->next = head; head = newNode.

◆ Doubly LL Insertion at kth position

Traverse to (k-1)th node. Set newNode->prev = current; newNode->next = current->next; update neighbours' prev/next pointers.

◆ Stack as Linked List

Push = insert at head (O(1)); Pop = delete from head (O(1)). No overflow (dynamic).

◆ Queue as Linked List

Enqueue = insert at tail; Dequeue = delete from head. Maintain both head and tail pointers.

◆ Circular Linked List

Last node's next points back to head (not NULL). Used in round-robin scheduling, circular buffers.

■ *Exam Tip: This unit has the highest total appearances. The combination question 'Implement Stack/Queue using Linked List' is critical for Section A. Draw node diagrams to support your written answer.*

Unit 6 — Sorting

Syllabus Hours: 8 Hrs | ■ VERY HIGH | **Appeared in:** 2074, 2075, 2077, 2078, 2079, 2080, 2081

■ Syllabus Topics

- 6.1 Internal vs External Sorting
- 6.2 Comparison Sorts: Bubble Sort, Selection Sort, Insertion Sort, Shell Sort
- 6.3 Divide and Conquer: Merge Sort, Quick Sort, Heap Sort
- 6.4 Efficiency and Time Complexity of Sorting Algorithms

■ Most Repeated Questions (with Year & Section)

#	Question / Topic	Type	Years Asked
1	Shell Sort — sort given array (e.g., {82,73,12,39,26,88,2,9,60,41})	Both	2079,2080,2081
2	Quick Sort (Divide and Conquer) — hand trace with given array	Long (A)	2075,2079
3	Selection Sort — trace on given array	Short (B)	2078,2080
4	Bubble Sort — hand test with given array (ascending/descending)	Short (B)	2074,2077
5	Insertion Sort — write a program to implement	Short (B)	2080
6	Heap Sort / Create Max Heap from numbers	Short (B)	2079

■ Key Concepts You Must Know

◆ Bubble Sort — $O(n^2)$

Repeatedly swap adjacent elements if out of order. Best case O(n) with optimisation flag. Simple but slow.

◆ Selection Sort — $O(n^2)$

Find minimum in unsorted part, swap with first unsorted element. Always $O(n^2)$ — no best case optimisation.

◆ Insertion Sort — $O(n^2)$

Pick element, insert into correct position in sorted portion. Good for nearly sorted arrays. Best case O(n).

◆ Shell Sort — $O(n \log^2 n)$

Generalised insertion sort with gap sequence. Gap starts large (n/2), reduces by half each pass. Much faster than basic $O(n^2)$ sorts.

◆ Quick Sort — O(n log n) avg

Choose pivot, partition array (elements < pivot left, > pivot right), recursively sort partitions. Worst case $O(n^2)$ when pivot is always min/max.

◆ Merge Sort — O(n log n)

Divide array in half recursively, merge sorted halves. Always O(n log n), uses O(n) extra space.

■ *Exam Tip: For hand-trace questions, show EVERY pass in a table. Shell Sort is very popular — remember the gap sequence (n/2, n/4, ... 1). Quick Sort: show partitioning clearly around the pivot.*

Unit 7 — Searching and Hashing

Syllabus Hours: 6 Hrs | ■ VERY HIGH | **Appeared in:** 2074, 2075, 2077, 2078, 2079, 2080, 2081

■ Syllabus Topics

- 7.1 Sequential (Linear) Search — algorithm and time complexity
- 7.2 Binary Search — algorithm, requirement (sorted), time complexity O(log n)
- 7.3 Efficiency of Search Algorithms
- 7.4 Hashing: Hash Function, Hash Table
- 7.5 Collision Resolution: Linear Probing, Quadratic Probing, Double Hashing, Chaining
- 7.6 Rehashing

■ Most Repeated Questions (with Year & Section)

#	Question / Topic	Type	Years Asked
1	Hashing: define hash table, hash function, collision. Explain linear probing and quadratic probing with example.	Long	2074,2075,2077,2078,2079,2080,2081
2	Apply linear probing and double hashing to store keys into hash table of size N	Short (B)	2079,2081
3	Binary search — explain with example. Time complexity?	Short (B)	2075,2078,2079,2080
4	Write a program to implement binary search.	Short (B)	2079
5	Rehashing — explain with example.	Short (B)	2075,2081
6	Differentiate sequential search and binary search.	Short (B)	2074

■ Key Concepts You Must Know

◆ Binary Search — O(log n)

Requires SORTED array. Compare target with mid element. If target < mid: search left half; If target > mid: search right half. Repeat. Always O(log n).

◆ Hash Function

Maps a key to a table index. Common: h(k) = k % tableSize. Goal: uniform distribution, minimise collisions.

◆ Collision

Two keys map to the same index. Unavoidable in practice; resolved by probing or chaining.

◆ Linear Probing

If slot h(k) is occupied, try h(k)+1, h(k)+2, ... (mod tableSize). Simple but causes primary clustering.

◆ Quadratic Probing

Try $h(k)+1^2$, $h(k)+2^2$, $h(k)+3^2$, ... Reduces primary clustering but may cause secondary clustering.

◆ Double Hashing

Second hash function h2(k) determines step size. Probe sequence: h(k), h(k)+h2(k), h(k)+2*h2(k), ... Best collision resolution.

◆ Rehashing

When load factor (n/tableSize) exceeds threshold (~0.7), create a new larger table (usually 2x+1 size) and re-insert all elements.

■ *Exam Tip: Hashing is the MOST consistently repeated topic across all years. Always show your working step by step when probing. For a hash table of size 5 with h(x)=x%5, show each key insertion and which slot it occupies.*

Unit 8 — Trees and Graphs

Syllabus Hours: 8 Hrs | ■ MUST-KNOW | **Appeared in:** 2074, 2075, 2077, 2078, 2079, 2080, 2081

■ Syllabus Topics

- 8.1 Binary Tree — concepts, height, level, depth, types
- 8.2 Binary Search Tree (BST) — insertion, deletion, search, traversals
- 8.3 AVL Tree — balancing algorithm, LL/RR/LR/RL rotations
- 8.4 Applications of Trees (Huffman coding, expression trees, game trees)
- 8.5 Graphs — definition, representation (adjacency matrix/list), traversal
- 8.6 Minimum Spanning Tree: Kruskal's and Prim's algorithms
- 8.7 Shortest Path: Dijkstra's algorithm

■ Most Repeated Questions (with Year & Section)

#	Question / Topic	Type	Years Asked
1	AVL Tree: construction from given data (e.g., 24,12,8,15,35,30,57,40,45,78). Explain rotations.	Long (A)	2074,2077,2078,2079,2080,2081
2	Dijkstra's algorithm to find shortest path between two vertices.	Long (A)	2077,2080
3	MST using Prim's algorithm / Kruskal's algorithm on given graph.	Both	2074,2079,2081
4	BFS (Breadth First Search) traversal of graph — short note / explain.	Short (B)	2075,2079,2081
5	DFS and BFS traversal of graph with example.	Long (A)	2075
6	BST — program for insertion and deletion.	Long (A)	2078
7	Types of binary tree. Compare between them.	Short (B)	2074
8	Tree traversals — Pre-order, In-order, Post-order. Difference.	Short (B)	2074,2075
9	What is minimum spanning tree? Explain applications.	Short (B)	2074,2077,2081

■ Key Concepts You Must Know

◆ AVL Tree

Self-balancing BST. Balance factor = height(left) - height(right). Must be -1, 0, or +1. If |BF| > 1 → perform rotation. 4 cases: LL (right rotation), RR (left rotation), LR (left-right), RL (right-left).

◆ AVL LL Rotation

Right-heavy on left child. Simple right rotation: left child becomes new root, original root becomes its right child.

◆ AVL LR Rotation

Left child's right subtree is heavy. Two-step: left-rotate left child, then right-rotate root.

◆ BST Traversals

In-order (L→Root→R): gives sorted output. Pre-order (Root→L→R): for copying tree. Post-order (L→R→Root): for deleting tree.

◆ Prim's Algorithm (MST)

Greedy: start from any vertex. At each step, add the minimum-weight edge connecting visited to unvisited vertex. Continue until all vertices included.

◆ Kruskal's Algorithm (MST)

Sort all edges by weight. Add edge if it doesn't form a cycle (use Union-Find). Stop when MST has V-1 edges.

◆ Dijkstra's Algorithm

Single-source shortest path. Use a visited set and distance array (init all ∞, source=0). Greedily pick unvisited vertex with min distance. Relax its neighbours. Repeat.

◆ BFS vs DFS

BFS uses a Queue — explores level by level. DFS uses a Stack (or recursion) — goes deep first. BFS finds shortest path in unweighted graphs.

■ *Exam Tip: AVL Tree construction is the single most important long question. Memorise the 4 rotation cases with diagrams. For Dijkstra's, always show the distance table step-by-step. The data set 24,12,8,15,35,30,57,40,45,78 has been repeated for AVL trees — practise it until you can do it in your sleep!*

■ Exam Strategy & Revision Tips

Section A Focus (10 marks each – attempt any 2)

• AVL Tree construction is asked almost every year – practice with given data sets.
• Queue implementation (linear + circular) with program is a repeated long question.
• Linked List implementation of Stack and Queue appears across multiple years.
• Graph algorithms: be ready for either MST (Prim's/Kruskal's) or Dijkstra's.
• Stack with infix-to-postfix conversion + recursion often paired together.

Section B Focus (5 marks each – attempt any 8)

• Always attempt: Asymptotic notation (Big O), Hashing (linear probing), Binary Search.
• Sorting hand-trace: practice Shell Sort, Selection Sort, Bubble Sort on paper.
• Recursion programs: GCD, Fibonacci, TOH – short but guaranteed marks.
• Short notes: ADT, Circular Linked List, BFS/DFS, Priority Queue, Tail Recursion.
• Postfix evaluation using stack is a very consistent Section B question.

3-Day Revision Plan

• Day 1 – Unit 8 (AVL Tree, Graphs) + Unit 2 (Stack, Infix-Postfix).
• Day 2 – Unit 3 (Queue) + Unit 5 (Linked List operations, Stack/Queue as LL).
• Day 3 – Unit 6 (Sort tracing) + Unit 7 (Hashing) + Unit 1 (Big O) + Unit 4 (Recursion).

Programming Questions to Prepare

• Implement linear queue (array-based).
• Implement binary search.
• Implement insertion sort.
• Recursive GCD and Fibonacci.
• Linked list node insertion/deletion.

Chapter-wise Weightage Summary

Unit	Topic	Priority	Sec A Freq	Sec B Freq	Total Appearances
1	Introduction / Complexity	HIGH	-	6	6
2	Stack	MUST-KNOW	3	6	9
3	Queue	MUST-KNOW	5	5	10
4	Recursion	VERY HIGH	2	5	7
5	Lists / Linked List	MUST-KNOW	5	6	11
6	Sorting	VERY HIGH	3	6	9
7	Searching & Hashing	VERY HIGH	2	7	9
8	Trees & Graphs	MUST-KNOW	6	7	13

Good luck with your DSA exam! Focus on understanding concepts, not just memorizing — TU questions often ask you to apply the algorithm to new data.